



PROGRAMMING WITH PYTHON

Lekha A

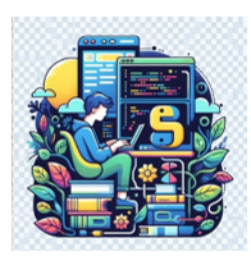
Computer Applications

PROGRAMMING WITH PYTHON

Python Basics and Standard Library Standard Library – sys, collections

Lekha A

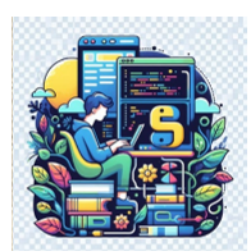
Computer Applications



PROGRAMMING WITH PYTHON

Standard Library - sys

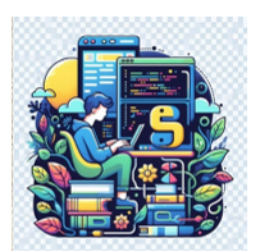
- The `sys` module in Python provides access to system-specific information and functions.
- It's a powerful tool for interacting with the operating environment and manipulating various aspects of the program's execution.



PROGRAMMING WITH PYTHON

Standard Library - sys

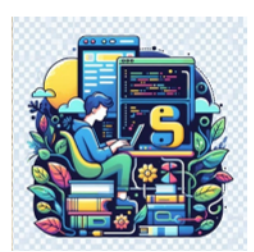
- System Information
 - `sys.version`
 - Returns the current Python version string.
 - `sys.platform`
 - Identifies the underlying operating system platform (e.g., 'win32', 'linux').



PROGRAMMING WITH PYTHON

Standard Library - sys

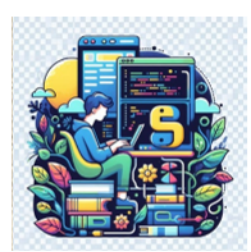
- `sys.argv`
 - Stores the command-line arguments passed to the program as a list.
- `sys.path`
 - Contains the list of directories searched for modules and packages.
- `sys.exit(code)`
 - Exits the program with a specific exit code.



PROGRAMMING WITH PYTHON

Standard Library - sys

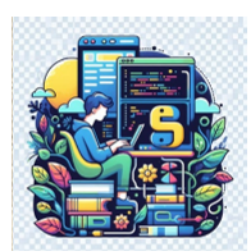
- File I/O
 - `sys.stdin`
 - Represents the standard input stream (keyboard by default).
 - `sys.stdout`
 - Represents the standard output stream (console by default).



PROGRAMMING WITH PYTHON

Standard Library - sys

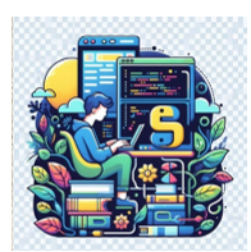
- `sys.stderr`
 - Represents the standard error stream (console for errors).



PROGRAMMING WITH PYTHON

Standard Library - sys

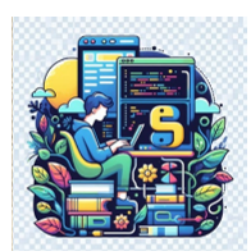
- Other Useful Functions
 - `sys.modules`
 - Provides a dictionary of all loaded modules.
 - `sys.getsizeof(object)`
 - Returns the size of an object in bytes.
 - `sys.traceback`
 - Contains information about the current call stack (useful for debugging).



PROGRAMMING WITH PYTHON

Standard Library – sys – Benefits of sys

- The sys module provides a powerful interface for
 - Tailoring Python programs to the underlying system and environment
 - Enhancing flexibility and control over program execution
 - Facilitating debugging and profiling
 - Creating dynamic and adaptable applications



PROGRAMMING WITH PYTHON

Standard Library – sys – Example

```
import sys
```

Accessing system information

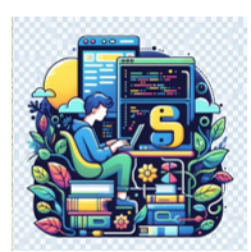
```
print("Python version:", sys.version)  
print("Operating system:", sys.platform)
```

```
Python version: 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)]  
Operating system: win32
```

Working with command-line arguments

```
print("Command-line arguments:", sys.argv)
```

```
Command-line arguments: ['C:\\Users\\dell\\AppData\\Local\\Programs\\Python\\Python312\\Lib\\site-packages\\  
1\\AppData\\Roaming\\jupyter\\runtime\\kernel-40e13804-5d24-4e17-8a51-150e116b9982.json']
```



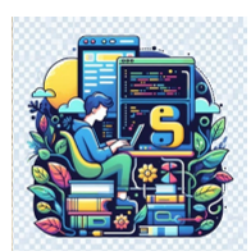
PROGRAMMING WITH PYTHON

Standard Library – sys – Example

- Displaying module search path

```
print("Module search path:", sys.path)
```

```
Module search path: ['E:\\MCA\\Notes_MCA\\FirstYear\\1Sem\\Computer Applications\\Python\\1Sem_\\Programs\\Python\\Python312\\python312.zip', 'C:\\Users\\dell\\AppData\\Local\\Programs\\Python\\Python312\\Lib', 'C:\\Users\\dell\\AppData\\Local\\Programs\\Python\\Python312\\Python312\\Lib\\site-packages', 'C:\\Users\\dell\\AppData\\Local\\Programs\\Python\\Python312\\ocal\\Programs\\Python\\Python312\\Lib\\site-packages\\win32\\lib', 'C:\\Users\\dell\\AppData\\thonwin']
```



PROGRAMMING WITH PYTHON

Standard Library – sys – Example

Simulating command-line arguments

```
sys.argv = ['script_name.py', 'Lekha']
```

Using sys.exit()

```
if len(sys.argv) < 2:  
    print("Please provide a name as a command-line argument.")  
    sys.exit(1)  
else:  
    name = sys.argv[1]  
    print("Hello,", name)
```

Hello, Lekha

Simulating command-line arguments

```
sys.argv = ['script_name.py']
```

Using sys.exit()

```
if len(sys.argv) < 2:  
    print("Please provide a name as a command-line argument.")  
    sys.exit(1)  
else:  
    name = sys.argv[1]  
    print("Hello,", name)
```

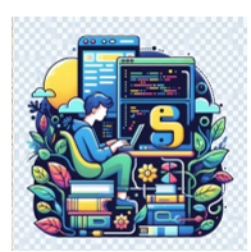
Please provide a name as a command-line argument.

An exception has occurred, use %tb to see the full traceback.

SystemExit: 1

C:\Users\dell\AppData\Local\Programs\Python\Python312\Lib\site-packa
or Ctrl-D.

warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)



PROGRAMMING WITH PYTHON

Standard Library – sys – Example

- **sys.stdin:** Represents the standard input stream (keyboard by default)

```
print("Please enter your name:")  
user_input = sys.stdin.readline().rstrip('\n')
```

Please enter your name:

- **sys.stdout:** Represents the standard output stream (console by default)

```
sys.stdout.write("Hello, ")  
sys.stdout.write(user_input + "\n")
```

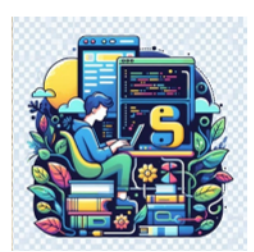
Hello,

1

- **sys.stderr:** Represents the standard error stream (console for errors)

```
sys.stderr.write("This is an error message.\n")
```

This is an error message.



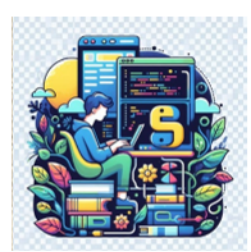
PROGRAMMING WITH PYTHON

Standard Library – sys – Example

sys.modules: Provides a dictionary of all loaded modules

```
loaded_modules = sys.modules
print("Loaded Modules:")
for module in loaded_modules:
    print(module)
```

```
Loaded Modules:
sys
builtins
_frozen_importlib
_imp
_thread
_warnings
_weakref
winreg
_io
marshal
nt
_frozen_importlib_external
time
zipimport
_codecs
codecs
encodings.aliases
```



PROGRAMMING WITH PYTHON

Standard Library – sys – Example

• **sys.getsizeof(object):** Returns the size of an object in bytes

```
num = 42
size_of_num = sys.getsizeof(num)
print(f"Size of 'num' object: {size_of_num} bytes")
```

Size of 'num' object: 28 bytes

sys.traceback: Contains information about the current call stack (useful for debugging)

```
try:
    result = math.exp(1000)
except Exception as e:
    exc_type, exc_value, exc_traceback = sys.exc_info()
    print("Traceback information:")
    traceback.print_tb(exc_traceback)
    print(f"Exception Type: {exc_type}")
    print(f"Exception Value: {exc_value}")
```

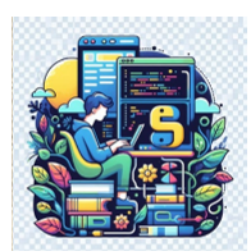
Traceback information:

Exception Type: <class 'OverflowError'>

Exception Value: math range error

File "C:\Users\dell\AppData\Local\Temp\ipykernel_30604\2173135176.py", line 2, in <module>

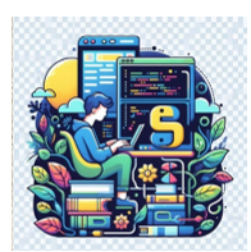
```
    result = math.exp(1000)
    ^^^^^^^^^^^^^^^^^^^
```



PROGRAMMING WITH PYTHON

Standard Library - collections

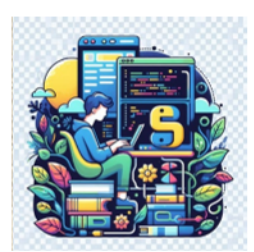
- The collections module in Python provides various specialized container datatypes beyond the built-in ones such as lists, tuples, dictionaries, and sets.



PROGRAMMING WITH PYTHON

Standard Library - collections

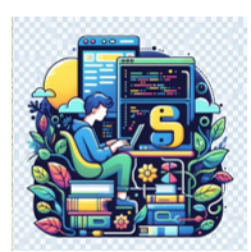
- `namedtuple()`
 - factory function for creating tuple subclasses with named fields
 - Allows creating tuple-like objects with named fields, enhancing code readability.



PROGRAMMING WITH PYTHON

Standard Library - collections

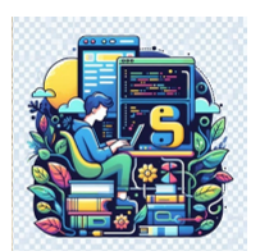
- deque
 - list-like container with fast appends and pops on either end
 - Provides an efficient double-ended queue with fast appends and pops from both ends.
- ChainMap
 - dict-like class for creating a single view of multiple mappings



PROGRAMMING WITH PYTHON

Standard Library - collections

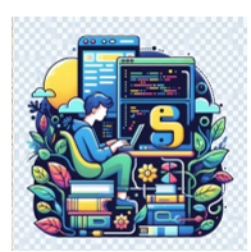
- Counter
 - dict subclass for counting hashable objects.
 - It simplifies counting hashable objects in collections.
- OrderedDict
 - dict subclass that remembers the order entries were added.



PROGRAMMING WITH PYTHON

Standard Library - collections

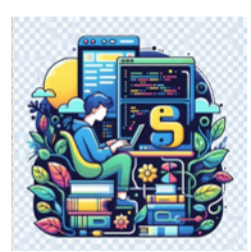
- defaultdict
 - dict subclass that calls a factory function to supply missing values for avoiding key errors
- UserDict
 - wrapper around dictionary objects for easier dict subclassing



PROGRAMMING WITH PYTHON

Standard Library - collections

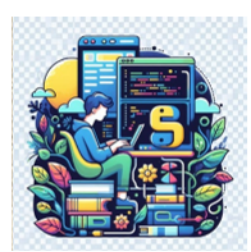
- UserList
 - wrapper around list objects for easier list subclassing
- UserString
 - wrapper around string objects for easier string subclassing



PROGRAMMING WITH PYTHON

Standard Library – collections - Benefits

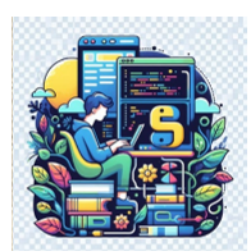
- Specialized Data Structures
 - The module offers specialized data structures that are highly optimized for specific use cases, providing improved performance and functionality compared to standard data types



PROGRAMMING WITH PYTHON

Standard Library – collections - Benefits

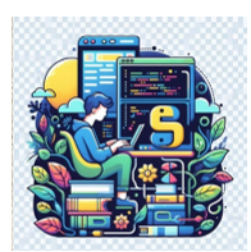
- Enhanced Functionality
 - These specialized data structures offer additional functionalities tailored to specific needs, making it easier to work with certain types of data or handle particular scenarios efficiently.
 - deque for efficiently handling queues or stacks.
 - Counter for quick frequency counting.
 - OrderedDict for maintaining insertion order in dictionaries



PROGRAMMING WITH PYTHON

Standard Library – collections - Benefits

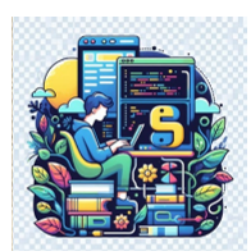
- Improved Performance
 - The data structures in the collections module are optimized for performance in specific scenarios.
 - deque provides faster operations for appending and popping elements from both ends compared to lists.



PROGRAMMING WITH PYTHON

Standard Library – collections - Benefits

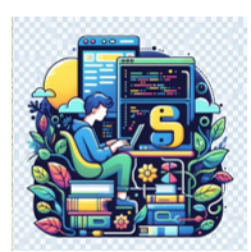
- Improved Performance
 - The data structures in the collections module are optimized for performance in specific scenarios.
 - deque provides faster operations for appending and popping elements from both ends compared to lists.
 - Counter efficiently counts occurrences of elements in an iterable.
 - defaultdict avoids key error exceptions by providing default values.



PROGRAMMING WITH PYTHON

Standard Library – collections - Benefits

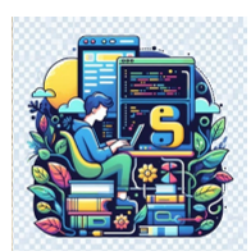
- Readability and Maintainability
 - Certain data structures like namedtuple and defaultdict can improve code readability by providing named fields or default values, enhancing code maintainability and comprehension.



PROGRAMMING WITH PYTHON

Standard Library – collections - Benefits

- Ease of Use
 - The data structures in the collections module are easy to use and integrate into the Python code.
 - They provide clean, concise solutions for specific data management scenarios, reducing the need for writing custom implementations.



PROGRAMMING WITH PYTHON

Standard Library – collections - Examples

• counter

```
from collections import Counter
```

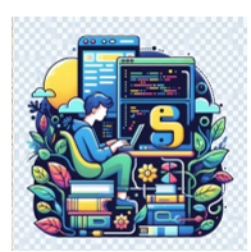
Count occurrences of elements in a list

```
my_list = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']  
counter = Counter(my_list)  
print(counter)
```

```
Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

```
Counter("Python is a great language")
```

```
Counter({' ': 4,  
        'a': 4,  
        'g': 3,  
        't': 2,  
        'n': 2,  
        'e': 2,  
        'P': 1,  
        'y': 1,  
        'h': 1,  
        'o': 1,  
        'i': 1,  
        's': 1,  
        'r': 1,  
        'l': 1,  
        'u': 1})
```

PROGRAMMING WITH PYTHON

Standard Library – collections - Examples

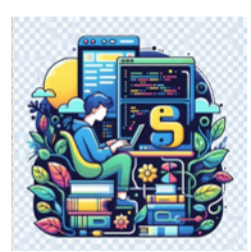
- Default Dictionary

```
from collections import defaultdict
```

```
def def_values():  
    return 'Not Existing'
```

```
d2=defaultdict(def_values)  
d2['a']=1  
d2['b']=1  
print(d2['a'],d2['b'],d2['c'])  
print(type(d2))  
print(d2)
```

```
1 1 Not Existing  
<class 'collections.defaultdict'>  
defaultdict(<function def_values at 0x0000020E1BE6AA20>, {'a': 1, 'b': 1, 'c': 'Not Existing'})
```



PROGRAMMING WITH PYTHON

Standard Library – collections - Examples

```
int_dict = defaultdict(int)
int_dict['a'] = 1
int_dict['b'] += 1
print(int_dict)
int_dict['c']
```

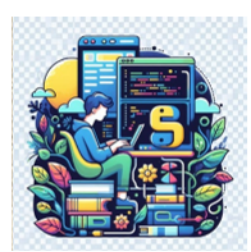
```
defaultdict(<class 'int'>, {'a': 1, 'b': 1})
```

0

```
int_dict = defaultdict(float)
int_dict['a'] = 1
int_dict['b'] += 1
print(int_dict)
int_dict['c']
```

```
defaultdict(<class 'float'>, {'a': 1, 'b': 1.0})
```

0.0



PROGRAMMING WITH PYTHON

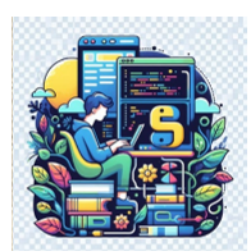
Standard Library – collections - Examples

- ```
int_dict = defaultdict(complex)
int_dict['a'] = 1
int_dict['b'] += 1
print(int_dict)
int_dict['c']
```

```
defaultdict(<class 'complex'>, {'a': 1, 'b': (1+0j)})
0j
```

```
int_dict = defaultdict(list)
int_dict['a'] = 1
int_dict['b'] = 2
print(int_dict)
int_dict['c']
```

```
defaultdict(<class 'list'>, {'a': 1, 'b': 2})
[]
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

```
int_dict = defaultdict(dict)
int_dict['a'] = 1
int_dict['b'] = 2
print(int_dict)
int_dict['c']
```

```
defaultdict(<class 'dict'>, {'a': 1, 'b': 2})
{}
```

```
int_dict = defaultdict(set)
int_dict['a'] = 1
int_dict['b'] = 2
print(int_dict)
int_dict['c']
```

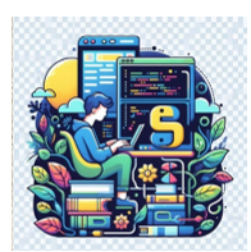
```
defaultdict(<class 'set'>, {'a': 1, 'b': 2})
set()
```

```
int_dict = defaultdict(tuple)
int_dict['a'] = 1
int_dict['b'] = 2
print(int_dict)
int_dict['c']
```

```
defaultdict(<class 'tuple'>, {'a': 1, 'b': 2})
()
```

```
int_dict = defaultdict(str)
int_dict['a'] = 1
int_dict['b'] = 2
print(int_dict)
int_dict['c']
```

```
defaultdict(<class 'str'>, {'a': 1, 'b': 2})
''
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

### • Ordered Dictionary

```
from collections import OrderedDict
```

```
ordered_dict = OrderedDict([("a", 1), ("b", 2), ("c", 3)])
print(ordered_dict)
```

```
OrderedDict({'a': 1, 'b': 2, 'c': 3})
```

### Move the value to the end

```
ordered_dict.move_to_end("b")
print(ordered_dict)
```

```
OrderedDict({'a': 1, 'c': 3, 'b': 2})
```

### Display keys in reverse order

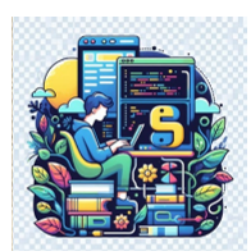
```
for key in reversed(ordered_dict):
 print(key)
```

```
b
c
a
```

Returns a view object that displays a list of a dictionary's (key, value) tuple pairs.

```
ordered_dict = OrderedDict([("a", 1), ("b", 2), ("c", 3)])
print(ordered_dict)
for key, value in ordered_dict.items():
 print(key, value)
```

```
OrderedDict({'a': 1, 'b': 2, 'c': 3})
a 1
b 2
c 3
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

```
ordered_dict = OrderedDict([("a", 1), ("b", 2), ("c", 3)])
print(ordered_dict)
item = ordered_dict.popitem()
print(item)
```

```
OrderedDict({'a': 1, 'b': 2, 'c': 3})
('c', 3)
```

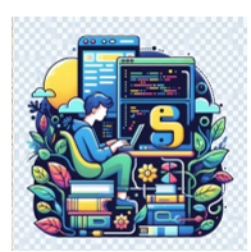
```
ordered_dict = OrderedDict([("a", 1), ("b", 2), ("c", 3)])
print(ordered_dict)
item = ordered_dict.popitem(last=True)
print(item)
```

```
OrderedDict({'a': 1, 'b': 2, 'c': 3})
('c', 3)
```

```
ordered_dict = OrderedDict([("a", 1), ("b", 2), ("c", 3)])
print(ordered_dict)
item = ordered_dict.popitem(last=False)
print(item)
```

```
OrderedDict({'a': 1, 'b': 2, 'c': 3})
('a', 1)
```





# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

### User Dictionary

```
from collections import UserDict
```

#### Creating a custom dictionary-like class by subclassing UserDict

```
class MyDict(UserDict):
 def get_keys(self):
 return [key.upper() for key in self.keys()]
```

#### Creating an instance of MyDict

```
custom_dict = MyDict({'a': 1, 'b': 2, 'c': 3})
```

#### Accessing dictionary keys and values

```
print(custom_dict)
print(custom_dict.get_keys())
```

```
{'a': 1, 'b': 2, 'c': 3}
['A', 'B', 'C']
```

#### Adding a new key-value pair

```
custom_dict['d'] = 4
print(custom_dict)
```

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

#### Modifying a value

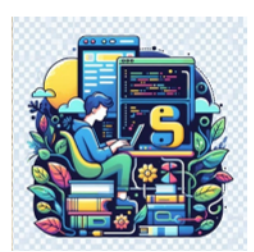
```
custom_dict['b'] = 20
print(custom_dict)
```

```
{'a': 1, 'b': 20, 'c': 3, 'd': 4}
```

#### Deleting a key-value pair

```
del custom_dict['c']
print(custom_dict)
```

```
{'a': 1, 'b': 20, 'd': 4}
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

### ChainMap

```
from collections import ChainMap
```

### Creating a chainmap

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
dict3 = {'e': 5, 'f': 6}
```

```
chain = ChainMap(dict1, dict2, dict3)
print(chain)
```

```
ChainMap({'a': 1, 'b': 2}, {'c': 3, 'd': 4}, {'e': 5, 'f': 6})
```

### Adding a dictionary at the front of the chainmap

```
new_chain = chain.new_child({'g': 5, 'h': 6})
print(new_chain)
```

```
ChainMap({'g': 5, 'h': 6}, {'a': 1, 'b': 2}, {'c': 3, 'd': 4}, {'e': 5, 'f': 6})
```

### Accessing values

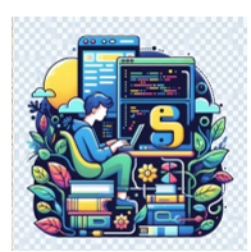
```
print(chain['b'])
print(chain['c'])
print(chain['d'])
```

```
2
3
4
```

```
value = chain.get('b')
print(value)
```

```
2
```





# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

Returns a new view of all the mappings in the chain except for the first one.

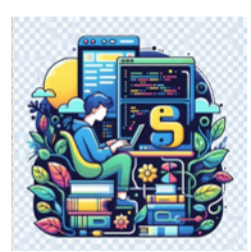
```
parent_maps = new_chain.parents
print(parent_maps)
```

```
ChainMap({'a': 1, 'b': 2}, {'c': 3, 'd': 4}, {'e': 5, 'f': 6})
```

Returns a view object that displays a list of (key, value) tuple pairs from all mappings in the chain.

```
for key, value in chain.items():
 print(key, value)
```

```
e 5
f 6
c 3
d 4
a 1
b 2
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

Returns a view object that displays a list of keys from all mappings in the chain.

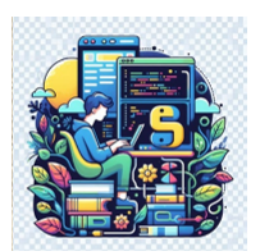
```
for key in chain.keys():
 print(key)
```

e  
f  
c  
d  
a  
b

Returns a view object that displays a list of values from all mappings in the chain.

```
for value in chain.values():
 print(value)
```

5  
6  
3  
4  
1  
2



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

### Deque

```
from collections import deque
```

### Create a deque

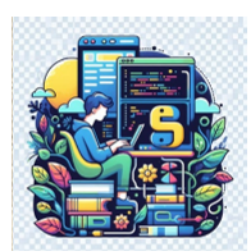
```
my_deque = deque([1, 2, 3])
```

```
my_deque.append(4) ##Insert at rear
my_deque.appendleft(0) ##Insert at front
print(my_deque)
```

```
deque([0, 1, 2, 3, 4])
```

```
my_deque.popleft() ##Delete from front
print(my_deque)
my_deque.pop() ##Delete from rear
print(my_deque)
```

```
deque([1, 2, 3, 4])
deque([1, 2, 3])
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

### • Named Tuple

```
from collections import namedtuple
```

### Create a named tuple

```
Person = namedtuple('Person', ['name', 'age'])
person1 = Person(name='Alice', age=30)
person2=Person('Frank',30)
print(person1.name, person1.age)
print(person1)
print(person2)
```

Alice 30

Person(name='Alice', age=30)

Person(name='Frank', age=30)

### Unpacking

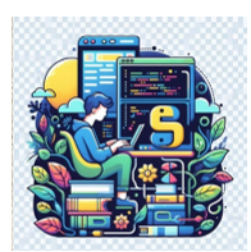
```
n,a=person2
print(n, a)
```

Frank 30

### Replacing a value

```
person3 = person1._replace(name='Neville')
print(person3)
```

Person(name='Neville', age=30)



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

### User List

```
from collections import UserList
```

### Create a Custom List class from User List

```
class CustomList(UserList):
 def append_twice(self, value):
 self.data.append(value)
 self.data.append(value)
 def pop_twice(self, pos):
 self.data.pop(pos)
 self.data.pop(pos)
```

### Create an instance of Custom List

```
custom_list = CustomList([1, 2, 3, 4, 5])
```

### Inherited Methods

```
custom_list.append(4)
print(custom_list)
```

```
[1, 2, 3, 4, 5, 4]
```

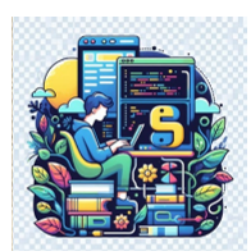
```
custom_list.extend([5, 6, 7])
print(custom_list)
```

```
[1, 2, 3, 4, 5, 4, 5, 6, 7]
```

```
custom_list.insert(2, 10)
print(custom_list)
```

```
[1, 2, 10, 3, 4, 5, 4, 5, 6, 7]
```

```
item = custom_list.pop(2)
print(item)
print(custom_list)
```



# PROGRAMMING WITH PYTHON

## Standard Library – collections - Examples

- ```
custom_list.remove(3)  
print(custom_list)
```

```
[1, 2, 4, 5, 4, 5, 6, 7]
```

```
custom_list.reverse()  
print(custom_list)
```

```
[7, 6, 5, 4, 5, 4, 2, 1]
```

```
custom_list.sort()  
print(custom_list)
```

```
[1, 2, 4, 4, 5, 5, 6, 7]
```

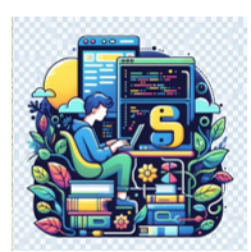
Use the custom method added in the subclass

```
custom_list.append_twice(5)  
print(custom_list)
```

```
[1, 2, 4, 4, 5, 5, 6, 7, 5, 5]
```

```
custom_list.pop_twice(5)  
print(custom_list)
```

```
[1, 2, 4, 4, 5, 7, 5, 5]
```



PROGRAMMING WITH PYTHON

Standard Library – collections - Examples

User String

```
from collections import UserString
```

Creating a custom string-like class by subclassing UserString

```
class MyString(UserString):  
    def append_exclamation(self):  
        # Custom method to append '!' to the string  
        return self.data + '!'
```

Creating an instance of MyString

```
custom_string = MyString("hello, world")
```

Accessing values

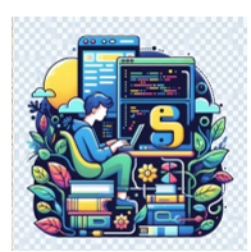
```
print(custom_string)
```

hello, world

Use the custom method added in subclass

```
modified_string = custom_string.append_exclamation()  
print(modified_string)
```

hello, world!



PROGRAMMING WITH PYTHON

Standard Library – collections - Examples

Demonstrating immutability

```
try:
    custom_string[0] = 'J'
except TypeError as e:
    print(f"TypeError: {e}")
```

TypeError: 'MyString' object does not support item assignment

Use Inherited Methods

```
capitalized_string = custom_string.capitalize()
print(capitalized_string)
```

Hello, world

```
lower_string = custom_string.lower()
print(lower_string)
```

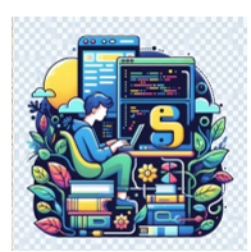
hello, world

```
upper_string = custom_string.upper()
print(upper_string)
```

HELLO, WORLD

```
title_string = custom_string.title()
print(title_string)
```

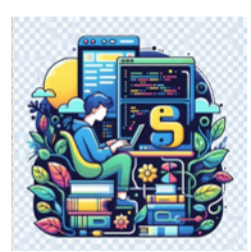
Hello, World



PROGRAMMING WITH PYTHON

Recap

- sys
 - System Information
 - File I/O
 - Modules
 - Size
 - Traceback
- Benefits



PROGRAMMING WITH PYTHON

Recap

- Collections
 - namedtuple()
 - deque
 - ChainMap
 - Counter
 - OrderedDict
- OrderedDict
 - defaultdict
 - UserDict
 - UserList
 - UserString
- Benefits



THANK YOU

Lekha A
Computer Applications